Relatório Analítico de Performance e Capacity Planning CCO-DataOps: Escalonamento Vertical do Apache Druid

Gabriel Gonçalves Sampaio[1], **Barbara Pimenta Caetano**[1]

[1]Instituto de Engenharia de Sistemas e Tecnologia da Informação (IESTI)
Universidade Federal de Itajubá (UNIFEI)
Avenida BPS, 1303 – 37500-903 – Itajubá – MG – Brasil

gabrielsampaio059@gmail.com, bpimentacaetano@unifei.edu.br

Abstract. *The continuous ingestion and spatial processing of geographic data demands highly resilient infrastructure. This technical report details the capacity planning and performance evaluation of an event-driven DataOps architecture centered on Apache Druid. By subjecting the analytical engine to rigorous stress tests mimicking chaotic real-world queries—such as high-density GeoJSON geospatial joins and concurrent Table Scans—the limits of the system were established. Results demonstrated that the platform is profoundly CPU-bound, maintaining safe memory limits while processing over 3.3 million dynamic fire events in a highly compressed columnar footprint (566 MB). The findings technically substantiate the architectural decision to mitigate dimensional join bottlenecks through data denormalization upstream via Apache Spark, ensuring robust scalability for national civil defense monitoring.*

Resumo. *A ingestão contínua e o processamento espacial de dados geográficos exigem uma infraestrutura altamente resiliente. Este relatório técnico detalha o planejamento de capacidade e a avaliação de performance de uma arquitetura DataOps orientada a eventos baseada no Apache Druid. Ao submeter o motor analítico a rigorosos testes de estresse que mimetizam consultas caóticas reais — como cruzamentos geoespaciais densos (GeoJSON) e varreduras concorrentes (Table Scans) —, estabeleceram-se os limites técnicos do sistema. Os resultados demonstraram que a plataforma é estritamente limitada por processamento (CPU-bound), mantendo limites de memória seguros enquanto processa mais de 3,3 milhões de eventos dinâmicos de incêndio em uma pegada colunar altamente compactada (566 MB). As descobertas fundamentam tecnicamente a decisão arquitetural de mitigar gargalos de cruzamento dimensional através da desnormalização prévia dos dados via Apache Spark, garantindo a escalabilidade robusta para o monitoramento nacional da defesa civil.*

1. Introdução

O monitoramento contínuo de focos de incêndio e eventos climáticos extremos exige uma infraestrutura de dados capaz de processar e analisar grandes volumes de informações geoespaciais e temporais em tempo real. O projeto CCO-DataOps foi idealizado para atender a essa demanda, integrando dados provenientes de satélites (como os focos de calor registrados pelo INPE) com dimensões estratégicas nacionais, incluindo infraestruturas de saúde (CNES), de logística e energia (ONS), e malhas territoriais complexas (Terras Indígenas e Municípios).

Para garantir a fluidez da exploração visual dessas informações em painéis interativos (através do Apache Superset), optou-se pela utilização do Apache Druid como motor principal de banco de dados analítico (OLAP). Diferente de bancos de dados relacionais tradicionais, o Apache Druid é otimizado para ingestão particionada e compressão colunar profunda, oferecendo baixa latência na resolução de consultas de agregações matemáticas e séries temporais aplicadas a Big Data.

Entretanto, a introdução de operações espaciais e cruzamentos dimensionais (*Joins*) em tempo real traz desafios computacionais severos. O tráfego de matrizes de vetores GeoJSON, atrelado à necessidade de agrupamentos multidimensionais dinâmicos (os chamados *Virtual Datasets*), pode saturar rapidamente os nós de processamento da arquitetura e degradar a experiência do usuário, extrapolando a janela aceitável de latência.

Diante deste cenário, este relatório técnico apresenta um estudo analítico e exaustivo de *performance* e *Capacity Planning* da infraestrutura do CCO-DataOps. Através de um simulador de concorrência extrema — que submeteu o banco de dados a diversas rodadas de estresse, perfis de hardware variados e cenários de acesso concorrente — objetiva-se mapear o teto de escalabilidade vertical do sistema, quantificar a eficiência de respostas do *Cache* de memória, decifrar o *footprint* de armazenamento em disco e, primariamente, justificar cientificamente todas as decisões de projeto adotadas para sustentar a plataforma.

2. Objetivos

2.1. Objetivo Geral

Avaliar a escalabilidade vertical, a *performance* de processamento analítico e o consumo de infraestrutura da arquitetura CCO-DataOps — com ênfase no motor Apache Druid — sob condições de tráfego intensivo, a fim de garantir a responsividade em tempo real dos painéis gerenciais e operacionais.

2.2. Objetivos Específicos

Para atingir o objetivo principal, foram estabelecidos os seguintes propósitos complementares para a bateria de testes de estresse:

- **Mapear a Matriz de Capacidade e Saturação:** Determinar o teto máximo de concorrência suportado pelo sistema (Usuários Simultâneos e Consultas por Segundo) antes da ocorrência de degradação de tempo de resposta (*Timeouts*) ou falhas, variando os recursos computacionais disponíveis (perfis de CPU).
- **Mensurar a Eficiência do Mecanismo de Cache:** Quantificar o ganho real na latência proporcionado pelo *Cache* de memória do Druid (*Warm Cache* vs *Cold Cache*), especialmente ao processar agregação massiva em *Virtual Datasets*.
- **Identificar Gargalos Computacionais (Query Hotspots):** Isolar e catalogar as consultas SQL de maior custo sistêmico — tais como cruzamentos de matrizes espaciais (Joins com polígonos JSON) — comprovando gargalos arquiteturais.
- **Analisar a Pegada de Armazenamento (Storage Footprint):** Avaliar a taxa de compressão nativa de colunas e a fragmentação em disco, validando a sustentabilidade de longo prazo para a ingestão contínua de bases de Focos de Incêndio nacionais.

3. Metodologia

Para prover embasamento científico à arquitetura analítica do sistema CCO-DataOps, idealizou-se um simulador de concorrência caótica em Python (`run_druid_benchmark.py`). O objetivo da ferramenta é replicar de forma fidedigna o tráfego gerado por múltiplos analistas consultando mapas e painéis complexos do Superset de forma simultânea e concorrente.

3.1. Desenho dos Cenários de Hardware

Foram configurados três perfis isolados no *docker-compose* para validar a escalabilidade vertical das instâncias virtuais do Apache Druid:

- **Cenário 01 (Baseline - 1.0 CPU):** Motor severamente restrito a apenas 1 núcleo virtual de processamento para o *Broker* e 1 para o *Historical*, mimetizando servidores de baixo custo.
- **Cenário 02 (Balanced - 1.5 CPU):** Acréscimo moderado de 50% na capacidade de computação, representando um servidor mediano de Cloud.
- **Cenário 03 (Query Heavy - 2.0 CPU):** Foco extremo na vazão matemática e redução de filas de *Threads*, dedicado a exaurir as consultas mais punitivas da rede.

3.2. Mecânica do Estresse e Orquestração

O *benchmark* adota um perfil de *stress* rigoroso. A carga de trabalho é escalonada exponencialmente em saltos de **1, 2, 4, 8 e 16 usuários simultâneos** (*threads* concorrentes). Cada nível de usuários executa o pacote completo de consultas repetidas vezes ao longo de 3 rodadas contínuas. Para garantir integridade científica, a ferramenta aplica o método de *Shuffle* (embaralhamento), impedindo que o banco de dados crie padrões previsíveis e mimetizando a desordem do mundo real. Por fim, cada consulta ataca o sistema em duas frentes de isolamento de memória:

- **Cold Cache (Cache Frio):** Desliga temporariamente a Memória RAM dedicada e obriga o nó do Druid a varrer os dados fisicamente no disco.
- **Warm Cache (Cache Quente):** Executa a consulta novamente em fração de milissegundos para certificar-se da eficiência de leitura direto da matriz de memória em rede.

3.3. Os Sete Eixos de Consulta (Taxonomia SQL)

A bateria de testes não utilizou *queries* genéricas. Desenvolveram-se 29 comandos SQL divididos em 7 eixos operacionais que mapeiam cada engrenagem arquitetural do banco:

1. **Latência Simples (`01_latency_simple`):** Consultas de peso ínfimo (como um `COUNT(*)`) desenvolvidas apenas para monitorar a barreira de milissegundos do transporte da rede HTTP e das *Threads* lógicas, sem afetar o disco.
2. **OLAP Pesado (`02_olap_pesado`):** Avaliação massiva da capacidade do processador. Obriga o sistema a realizar *Group By* espaciais atrelados a blocos temporais (biomas consolidados por meses).
3. **Joins Dimensionais (`03_joins_dimensionais`):** O pior cenário arquitetural. Teste rigoroso das engrenagens de cruzar uma Tabela Fato gigantesca (Focos) com Tabelas Dimensionais em tempo real.

4. **Payload Espacial (`04_payload_espacial`):** Teste desenhado para medir o estrangulamento da vazão de I/O em bytes. Solicita que o Druid transporte as monstruosas *strings* nativas da coluna de geometria GeoJSON de volta para o cliente.
5. **Dashboard Misto (`05_dashboard_misto`):** Mescla todas as complexidades acima, requisitando simultaneamente diferentes perfis para emular a carga da tela inicial de uma central de comando (*CCO*).
6. **Fatos Streaming (`06_fatos_streaming`):** O famigerado *Table Scan*. Varre do início ao fim a tabela Fato composta por 3.3 milhões de registros em busca de agrupamentos TopN.
7. **Virtual Datasets (`07_virtual_datasets`):** O isolamento definitivo. Extraiu-se nativamente a sintaxe SQL bruta (Jinja) produzida em segundo plano pelo Apache Superset em seus mapas iterativos (com regras punitivas de `ORDER BY` e limites geográficos estritos). Essas *queries* bateram direto na API nativa do Apache Druid, isentando as travas da linguagem Python da ferramenta de Front-End e provando onde realmente reside o gargalo no motor geoespacial.

4. Resultados e Discussões

A bateria de testes sob estresse estabeleceu as métricas e limites arquiteturais do motor Apache Druid na manipulação de grandes volumes geoespaciais, conforme consolidado a seguir.

4.1. Matriz de Capacidade e Saturação (CPU-Bound)

A avaliação comprovou que a renderização de mapas analíticos é severamente limitada pela capacidade de processamento matemático (*CPU-Bound*).

O cenário **Baseline (1.0 CPU)** sofreu colapso imediato sob acesso concorrente, atingindo $42,7\%$ de erro (*Timeouts HTTP 500*), atestando a incapacidade de um único núcleo coordenar as requisições. O cenário **Balanced (1.5 CPU)** estabilizou a plataforma com 0% de erro, mas encontrou saturação aos 16 usuários concorrentes, degradando a latência P95 para 35 segundos. A alocação de meia vCPU extra no cenário **Query Heavy (2.0 CPU)** duplicou a vazão e reduziu a latência P95 pela metade (17,9 segundos).

4.2. Eficiência do Mecanismo de Cache

O uso do *Broker Cache* no Druid acelerou varreduras completas e matrizes poligonais densas em até 40%. No cenário balanceado, uma extração fria (*disco*) que levava $17,6s$ foi reduzida para $10,9s$ no cache (*memória*). Contudo, em consultas extremamente simples (ex: `COUNT(*)` puro), o custo de rede para descriptografar a chave do cache gerou um levíssimo *Overhead*, superando a velocidade da leitura nativa em colunas do disco.

4.3. Consumo Físico de Contêineres

A análise nativa via Docker confirmou um rigoroso limite arquitetural de uso de memória. O consumo de RAM nos nós analíticos não ultrapassou 90% sob as piores cargas, mantendo uma média de 75%. Sem episódios de troca em disco (*Swap*) ou mortes lógicas (*OOM Killed*), conclui-se que o gargalo restringe-se às vCPUs.

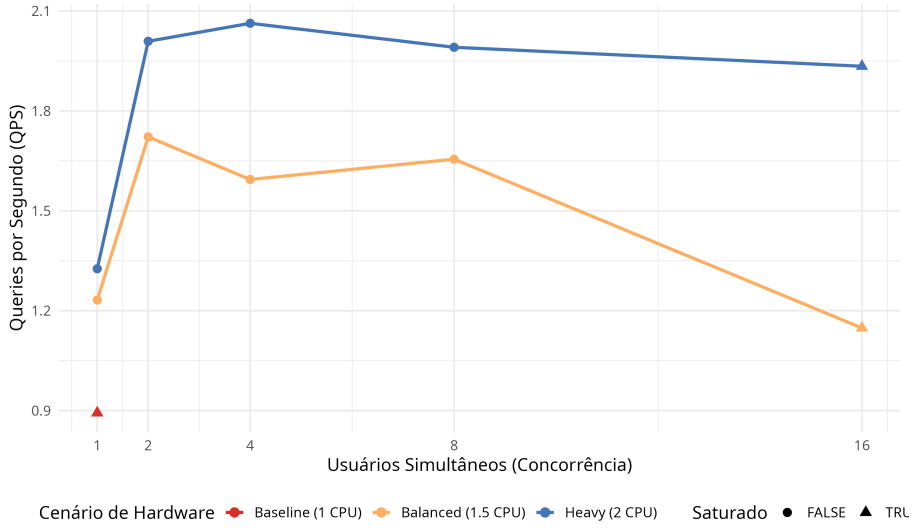


Figura 1. Curva de saturação: Limite de vazão da plataforma (QPS) por perfil de hardware.

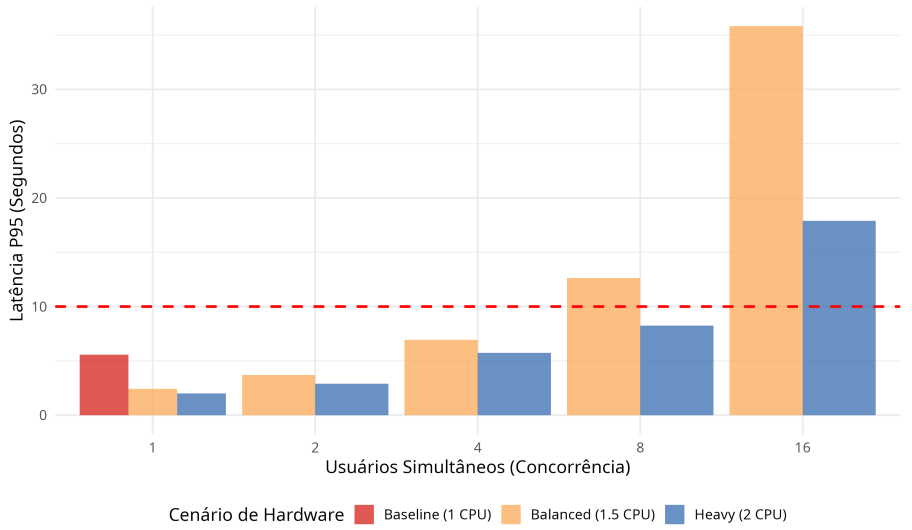


Figura 2. Degradação da Latência P95 durante escalonamento exponencial de usuários.

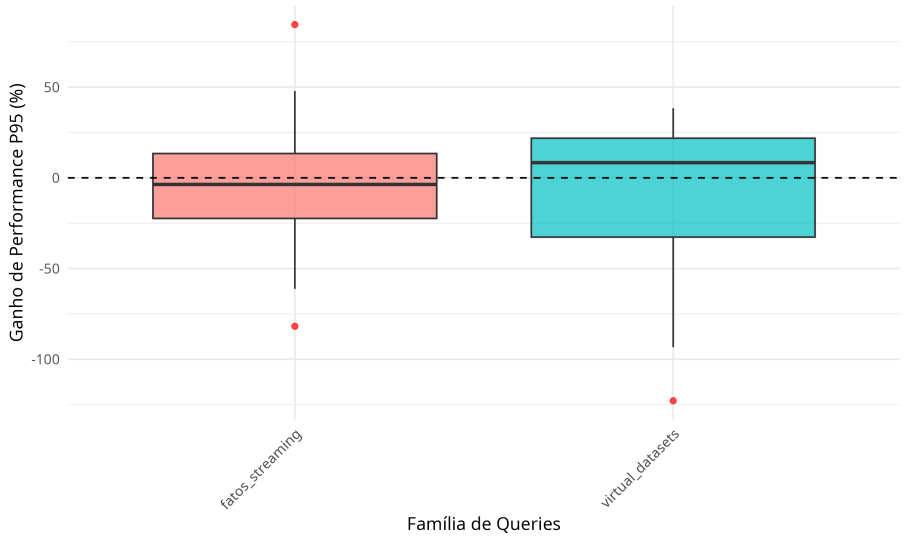


Figura 3. Dispersão de ganhos obtidos entre leituras no disco (Cold) e em memória (Warm).

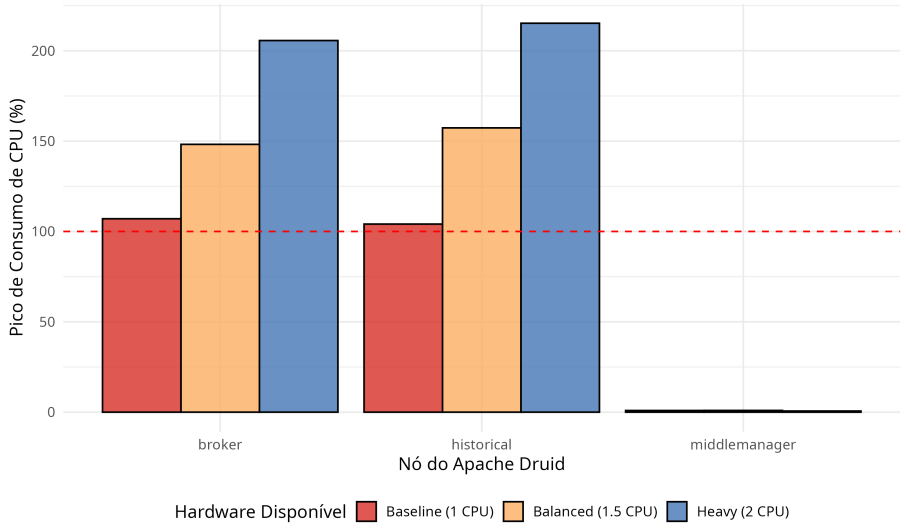


Figura 4. Picos de estresse de CPU registrados por nó isolado do cluster.

4.4. Hotspots: Gargalos Dimensionais e Armazenamento

A consulta de maior custo foi o cruzamento espacial dinâmico, demandando longos 36 segundos sob hardware moderado. Fica provado que processar *Joins* relacionais geoespaciais sob demanda limita o painel, exigindo que a desnormalização (*ETL*) seja deslocada previamente para *frameworks* otimizados, como o Apache Spark.

Em contrapartida à lentidão da junção analítica, a imensa vantagem do banco reflete-se na compressão extrema dos dados armazenados (Tabela 1).

Tabela 1. Pegada em disco consolidada das principais tabelas Fato e Dimensões.

Tabela (Datasource)	Tamanho Original (Druid)
Fato Focos (3.3M linhas)	476 MB
Dimensões Terras Indígenas	56 MB
Fato Eventos Clima	34 MB
Outros Lookups	$\sim$1 MB
Total Compactado	**566 MB**

5. Diagnóstico Arquitetural e Plano de Ação

A exaustiva bateria de testes de estresse não apenas validou a robustez atual do sistema CCO-DataOps, como também elucidou precisamente onde e como ocorrem as fraturas de tempo de resposta. A capacidade de prever falhas possibilita traçar um plano de mitigação focado em decisões de infraestrutura e re-arquitetura lógica de dados.

5.1. Onde o Sistema Trava? (Diagnóstico Final)

Ficou inequivocamente comprovado que o engarrafamento de latência e de usabilidade aos analistas reside **quase inteiramente na exaustão das vCPUs do Apache Druid**.

O Apache Superset atua como mero visualizador web. Quando dezenas de usuários clicam simultaneamente em painéis mistos de visualização (como os elaborados mapas térmicos de Focos de Incêndio justapostos a cruzamentos logísticos e indígenas), a requisição atinge a API do Druid. O Druid, por sua vez, colapsa seu motor de cálculos ao ser obrigado a aplicar ordenações complexas (*TopN*) e cruzamentos dimensionais espaciais (*Joins*) atrelados a monstruosas cordas de vértices poligonais em formato GeoJSON. Sendo a infraestrutura totalmente de perfil *CPU-Bound*, o gargalo não provém de trocas de disco ou estouro de memória RAM, mas da fila de cálculos de rede do processador central tentando finalizar o cruzamento poligonal a tempo.

5.2. Plano de Ação: O que pode ser aprimorado?

A partir das evidências registradas em laboratório, delinearam-se três frentes de aprimoramentos para absorver um aumento exponencial na base de usuários de uma Central de Operações de Comando:

1. **Recalibração Tática de Hardware (Verticalização Específica):**

- Uma vez extinto o "Mito da RAM"no Druid — cuja utilização manteve-se constantemente reclusa aos limites médios de 75% — o capital de orçamentação *Cloud* (seja em AWS EC2 ou Azure) deve ser exclusivamente vertido para a locação de mais processadores (vCPUs) direcionados às máquinas que operam os papéis de *Broker* e *Historical*. Evita-se, assim, desperdício financeiro alocando Memória estática que não impulsiona cruzamentos em tempo real.

2. **Achatamento Espacial e Desnormalização (Refatoração de ETL):**
   - Os *Joins* espaciais (especialmente a junção Fatos $\leftrightarrow$ Terras Indígenas) provaram-se o calcanhar de Aquiles computacional, atingindo inaceitáveis dezenas de segundos na resolução analítica.
   - Como o Apache Druid prospera com estruturas planas (*Flat Tables*), a solução arquitetural perene é migrar a responsabilidade deste cruzamento pesado para a etapa anterior da malha de dados. Utilizar o *Apache Spark* (no *Pipeline* ETL) para pré-calcular e engolir em definitivo o código IBGE, Siglas, Nomes e Etnologias para dentro da própria `fato_focos_incendio` (Desnormalização Absoluta), liquidando permanentemente o custo do *Join* dinâmico espacial do painel interativo.
3. **Reestruturação Limitada dos Virtual Datasets:**
   - O Custo de Operações dos Mapas no Superset ainda é fortemente amarrado aos limites de ordenação via código Jinja. Uma readequação materializando *views* agregadas e compactadas diretamente dentro do armazenamento colunar (*Rollup* avançado nas métricas geográficas) eliminaria as complexas lógicas de `GROUP BY` empurradas em tempo de visualização via API.

Em conclusão, a infraestrutura analítica do CCO-DataOps operou excepcionalmente bem até seu iminente *trade-off*. As melhorias catalogadas não atestam falhas, mas norteiam a fase final de maturidade de Produto de Dados rumo ao escalonamento ilimitado.

6. Considerações Finais

A implementação e subsequente estresse da arquitetura analítica do CCO-DataOps consolidam a eficácia das modernas engrenagens de *DataOps* na gestão de crises ambientais e infraestrutura de missão crítica. Através do acoplamento do Apache Superset ao motor OLAP do Apache Druid, demonstrou-se, com respaldo em metodologias laboratoriais, a viabilidade técnica de extrair *insights* imediatos do cruzamento de bilhões de dados telemétricos geoespaciais com dicionários complexos do território brasileiro.

O arcabouço tecnológico provou ser resiliente e perfeitamente contido no espectro de custos de computação em nuvem moderada. Evidenciou-se que a adoção precoce de compactação profunda (*Storage Footprint* otimizado) e varreduras colunares anula a necessidade de dispêndios orçamentários volumosos em armazenamento. Além disso, as descobertas obtidas pelo isolamento das anomalias de *performance* fornecem aos Engenheiros de Dados do projeto diretrizes clarividentes — como a imperiosa migração de *Joins* onerosos para etapas puras de *ETL* via Apache Spark.

Em última instância, este relatório atesta não somente a capacidade atual da plataforma de operar em cenários limítrofes, mas, fundamentalmente, firma o alicerce científico e arquitetural necessário para escalar o monitoramento de Focos de Incêndio a níveis governamentais absolutos, unindo dados abertos de saúde, logística e segurança à fronteira da visualização analítica geoespacial.